Topic 10: Protocols and Extensions

Task 1: Payroll with protocols

```
import Foundation
protocol Payable {
    var name: String { get }
    func monthlyPay() -> Double
}
extension Payable {
    func payslip() -> String {
        return "\(name): Rs. \(String(format: "%.2f", monthlyPay())) this month"
    }
}
struct FullTimeEmployee: Payable {
    var name: String
    var annualSalary: Double

    func monthlyPay() -> Double {
        return annualSalary / 12
    }
}
struct Intern: Payable {
    var name: String
    var monthlyStipend: Double

    func monthlyPay() -> Double {
        return monthlyStipend
    }
}
```

```
var people: [any Payable] = [
    FullTimeEmployee(name: "Asha", annualSalary: 600000),
    Intern(name: "Ravi", monthlyStipend: 15000)
]

for person in people {
    print(person.payslip())
}
struct Contractor: Payable {
    var name: String
    var hourlyRate: Double
    var hoursWorked: Double

    func monthlyPay() -> Double {
        return hourlyRate * hoursWorked
    }
}
people.append(Contractor(name: "Meena", hourlyRate: 800, hoursWorked: 120))
print("--- After adding the contractor ---")
for person in people {
    print(person.payslip())
}
```

```
Asha: Rs. 50000.00 this month
Ravi: Rs. 15000.00 this month
--- After adding the contractor ---
Asha: Rs. 50000.00 this month
Ravi: Rs. 15000.00 this month
Meena: Rs. 96000.00 this month
```

Task 2: Extend existing types

```
extension Int {
    var isEven: Bool {
        return self % 2 == 0
    }
    func squared() -> Int {
        return self * self
    }
}
print(4.isEven)
print(5.isEven)
print(5.squared())
extension Array where Element == Int {
    var sum: Int {
        var total = 0
        for number in self {
            total += number
        }
        return total
    }
}
print([1, 2, 3, 4, 5].sum)
extension String {
    var wordCount: Int {
        return self.split(separator: " ").count
    }
}
print("Swift is fun to learn".wordCount)
struct Version: Equatable {
```

```
    var major: Int
    var minor: Int
}
extension Version: Comparable {
    static func < (left: Version, right: Version) -> Bool {
        if left.major != right.major {
            return left.major < right.major
        }
        return left.minor < right.minor
    }
}
print(Version(major: 1, minor: 9) < Version(major: 2, minor: 0))
let versions = [
    Version(major: 2, minor: 0),
    Version(major: 1, minor: 9),
    Version(major: 1, minor: 2)
]
let sortedVersions = versions.sorted()
for version in sortedVersions {
    print("\(version.major).\(version.minor)")
}
```

```
true
false
25
15
5
true
1.2
1.9
2.0
```

Task 3: Protocol composition

```
protocol Flyable {
    func fly()
}

protocol Swimmable {
    func swim()
}
struct Duck: Flyable, Swimmable {
    func fly() { print("The duck flaps its wings and flies") }
    func swim() { print("The duck paddles across the pond") }
}
struct Eagle: Flyable {
    func fly() { print("The eagle soars high above") }
}
struct Fish: Swimmable {
    func swim() { print("The fish glides through the water") }
}
func showOff(_ animal: Flyable & Swimmable) {
    animal.fly()
    animal.swim()
}
showOff(Duck())
```

```
The duck flaps its wings and flies
The duck paddles across the pond
```